ОТЧЁТ О ЛАБОРАТОРНОЙ РАБОТЕ

Разработка и анализ прототипа сервиса анализа и визуализации транспортных потоков CityPulse с помощью вайб-кодинга (Cursor + ИИ-ассистент)

1. Цель и задачи работы

Цель работы — спроектировать и реализовать прототип модуля анализа и визуализации транспортных потоков системы CityPulse на основе выданного технического задания и ранее подготовленных UML-диаграмм, используя подход вайб-кодинга (быстрая итеративная разработка через диалог с ИИ), а затем провести аналитическую оценку качества полученного решения.

Основные задачи

- Проанализировать контекст (ТЗ, структурные и поведенческие UML-диаграммы CityPulse) и зафиксировать архитектуру модуля транспортных потоков.
- Разработать структуру данных и ключевые пользовательские сценарии для подсистемы RoadSegment / TrafficFlow / Alert / AnalyticsMetric / User.
- Реализовать прототип backend-сервиса на FastAPI и простой веб-интерфейс (Leaflet + Tailwind) для визуализации дорожной обстановки.
- Обеспечить готовность к развёртыванию в средах Replit/Cursor/Railway при помощи минимального, но полного набора конфигурационных файлов.
- Выполнить самоанализ: оценить производительность, качество кода, отсутствие очевидных багов и сценарии развёртывания.
- Сформулировать рефлексивное заключение о применении вайб-кодинга в разработке подобных сервисов.

2. Ссылки на материалы

Материал	Ссылка
Чат с ИИ-ассистентом	Cursor IDE — история сессии сохранена в репозитории chat)
Репозиторий проекта	https://github.com/egorr-gubanov/design_of_information_systems
Демонстрация работы	Видео-демонстрация: прилагается к отчёту (demo.mp4). Локальный запуск: http://localhost:8000

3. Архитектура и структура данных

3.1 Общая архитектура прототипа

Прототип реализует упрощённый модуль CityPulse с сохранением ключевых архитектурных принципов из ТЗ. Стек состоит из трёх компонентов:

- Backend: FastAPI + SQLModel (над SQLAlchemy) + SQLite
- Frontend: HTML-шаблон Jinja2 + Leaflet (CDN) + TailwindCSS (CDN) + Fetch API
- Аналитический модуль: services_analytics.py с расчётом congestion_index и генерацией алертов

Архитектурная схема (потоки данных)

DataIngestMock (имитация датчиков)	→	**FastAPI Backend (роутеры + сервисы)**	⇄	**SQLite DB (SQLModel ORM)**	←	**Web UI (Leaflet)**

POST /api/traffic/ingest → AnalyticsService → AlertService | GET /api/traffic/current, /api/alerts

3.2 Структура данных (ORM-модели)

Сущность	Ключевые атрибуты	Назначение
RoadSegment	id, name, geometry (GeoJSON), max_speed, length_meters	Дорожный участок с геометрией и скоростным режимом
TrafficFlow	segment_id (FK), timestamp, average_speed, vehicle_count, occupancy (0–100%)	Снимок транспортного потока от датчиков
Alert	segment_id, type (CONGESTION/ACCIDENT/HAZARD), severity (1–5), is_resolved	Инцидент/пробка с жизненным циклом (открыт/закрыт)
AnalyticsMetric	segment_id, time_window, congestion_index (0–100), average_flow, mean_delay_minutes	Агрегированные метрики за период
User	id, name, email, role (ADMIN/ANALYST/VIEWER), is_active	Пользователь с RBAC-ролью (заготовка для будущего)

3.3 Pydantic-схемы (DTO)

В файле schemas.py определены разделённые схемы Create/Read для каждой сущности, а также сервисные структуры:

- TrafficSnapshot — агрегированный ответ для карты (участок + последний поток + congestion_index + флаг алерта)
- TrafficIngestBatch — батч трафика для имитации потока от датчиков/Kafka

4. Пользовательские сценарии

Сценарий 1: Диспетчер мониторит ситуацию в реальном времени

Роль: VIEWER / диспетчер центра управления трафиком

1. Пользователь открывает веб-интерфейс модуля CityPulse.
2. Frontend инициализирует Leaflet-карту и выполняет запрос GET /api/traffic/current.
3. Backend для каждого RoadSegment собирает: последний TrafficFlow, последний AnalyticsMetric за час, наличие активного Alert.
4. Карта подсвечивает участки по congestion_index: зелёный — свободно, жёлтый — умеренная загруженность, красный — пробка. При клике отображаются скорость, интенсивность, алерт.
5. Каждые 15 секунд frontend опрашивает /api/traffic/current, /api/alerts, /api/metrics/summary через setInterval — квазиреальное время без WebSocket.

Сценарий 2: Аналитик строит краткий отчёт по участку

Роль: ANALYST

6. Аналитик смотрит на последние метрики через панель «Последние метрики».
7. Интерфейс запрашивает GET /api/metrics/summary?limit=5.
8. Backend возвращает список AnalyticsMetricRead с congestion_index и average_flow по сегментам.
9. Аналитик оценивает нагрузку и при необходимости выгружает данные через API (расширяемый шаг — экспорт в CSV).

Сценарий 3: Оператор реагирует на пробку

Роль: оператор смены

10. В систему поступает батч трафика через POST /api/traffic/ingest (каждый элемент — TrafficFlowCreate).
11. Backend сохраняет данные и пересчитывает метрики: низкая скорость + высокая occupancy → рост congestion_index.
12. Если congestion_index > 70 и средняя скорость < 20 км/ч — автоматически создаётся Alert типа CONGESTION с серьёзностью, зависящей от индекса.
13. Оператор видит алерт в боковой панели и при устранении вызывает POST /api/alerts/{id}/resolve.

5. Реализация прототипа

5.1 Backend (FastAPI + SQLModel)

Файл	Ответственность
main.py	Точка входа FastAPI: startup → init_db(), регистрация роутеров, статика, шаблоны, PORT из env
db.py	DATABASE_URL (SQLite → PostgreSQL через env), check_same_thread=False, init_db(), get_session()
models.py	ORM-модели: RoadSegment, TrafficFlow, Alert, AnalyticsMetric, User; перечисления AlertType, UserRole
schemas.py	Pydantic DTO: *Create / *Read для каждой сущности + TrafficSnapshot, TrafficIngestBatch
services_analytics.py	calculate_congestion_index(), ingest_traffic_batch(), _recompute_recent_metrics_for_segment(), _maybe_create_congestion_alert()
routers_segments.py	CRUD для RoadSegment: GET /, POST /, GET /{id}, DELETE /{id}
routers_traffic.py	POST /ingest (приём батча), GET /current (TrafficSnapshot для карты)
routers_alerts.py	GET /?only_active=true, POST / (ручное создание), POST /{id}/resolve
routers_metrics.py	GET /summary?limit=N — последние агрегированные метрики
seed_data.py	Автосидинг демо-данных: 3 сегмента СПб + тестовые TrafficFlow при первом старте

5.2 Frontend (Leaflet + TailwindCSS)

- templates/index.html: верхний бар с названием, двухколоночный контент — карта слева, панели «Активные алерты» и «Последние метрики» справа.
- static/js/app.js: инициализация Leaflet (центр — Санкт-Петербург, 59.93, 30.33), OpenStreetMap-тайлы (attributionControl: false для чистоты интерфейса), colorForCongestion(), parseGeometry(), setInterval(refreshAll, 15000).
- static/css/styles.css: системный шрифт, минимальная высота карты.

5.3 Конфигурация и деплой

- requirements.txt: fastapi==0.115.0, uvicorn[standard]==0.30.0, sqlmodel==0.0.37, jinja2==3.1.4, aiofiles==24.1.0, python-multipart
- Локальный запуск: uvicorn main:app --reload --host 0.0.0.0 --port 8000
- Railway / Render: uvicorn main:app --host 0.0.0.0 --port $PORT
- PythonAnywhere: ASGI-конфигурация main:app

6. Тестирование и анализ сервиса

6.1 Выявленные проблемы в ходе тестирования

В ходе разработки и тестирования было выявлено несколько проблем, которые потребовали итеративного исправления:

Проблема	Решение

Несовместимость sqlmodel==0.0.22 с pydantic==2.12.5 — падение при импорте моделей	Обновление sqlmodel до версии 0.0.37, совместимой с Pydantic v2
Отсутствие тестовых данных при первом старте — карта пустая	Создан seed_data.py с автосидингом 3 сегментов и тестовых TrafficFlow при старте
Атрибуция Leaflet (© OpenStreetMap) мешала чистоте интерфейса	Установлен attributionControl: false при инициализации карты; атрибуция убрана
zsh-глоббинг: символ ? в URL curl воспринимался как шаблон	URL при тестировании заключались в кавычки: curl 'http://...?param=value'
Конкурентный старт uvicorn на порту 8000 (старый процесс)	Сервер перезапущен на порту 8001; порт читается из переменной окружения PORT

6.2 Оценка производительности

В выбранной конфигурации (FastAPI + SQLite, отсутствие внешних сетевых вызовов и ML-моделей) прототип демонстрирует ожидаемо низкие задержки. Типовой запрос GET /api/traffic/current выполняется в пределах 20–50 мс при количестве сегментов, характерном для учебного прототипа (десятки–сотни записей).

Основное время уходит на чтение из SQLite и сериализацию Pydantic-ответа; сам FastAPI/uvicorn добавляет минимальный оверхед. Потребление памяти укладывается в сотни мегабайт даже при ограниченных ресурсах Replit.

Узкие места при промышленном масштабировании: отсутствие Kafka для потоковой обработки, упрощённая перерасчётная логика и использование SQLite вместо PostgreSQL+PostGIS. Для учебного стенда производительность более чем достаточна.

6.3 Анализ качества кода

- Код структурирован по слоям: модели и схемы разделены, бизнес-логика изолирована в services_analytics.py, роуты разбиты по доменам.
- Валидация Pydantic: ограничения на скорость, occupancy (0–100%), severity (1–5), enum-типы AlertType и UserRole — снижают риск «грязных» входных данных.
- Логика congestion_index и генерации алертов прозрачна, не содержит скрытых побочных эффектов — всё состояние в БД.
- Расширяемость: добавление WebSocket-канала или новой метрики не требует изменений существующих эндпоинтов.
- Остаточные риски: упрощённая транзакционность, отсутствие unit-тестов и полноценной аутентификации — принятые ограничения учебного прототипа.

6.4 Возможности развёртывания

Платформа	Инструкция
Replit / Cursor	pip install -r requirements.txt → Run: uvicorn main:app --reload --host 0.0.0.0 --port 8000
Railway / Render	Start command: uvicorn main:app --host 0.0.0.0 --port $PORT (порт из env)
PythonAnywhere	ASGI-конфигурация, приложение: main:app
PostgreSQL (прод)	Замена DATABASE_URL в db.py + добавление psycopg2 в requirements.txt; публичный API не меняется

7. Рефлексивное заключение

7.1 Сложности в ходе выполнения работы

Основным вызовом при генерации прототипа было соблюдение баланса между богатым ТЗ CityPulse (Kafka, PostGIS, ML-модель, WebSocket, сложная событийная

архитектура) и реалистичным объёмом кода для учебной работы. Пришлось сознательно абстрагироваться от инфраструктурных компонентов, сведя поток к REST-эндпоинту /api/traffic/ingest и периодическому опросу с фронтенда, но сохранить ключевые доменные сущности и логику индекса загруженности.

Дополнительная сложность — поддерживать чистую архитектуру в условиях итеративной генерации: каждый шаг требовал согласованности между моделями, схемами, роутами и фронтендом. Несовместимость пакетов (sqlmodel 0.0.22 vs pydantic 2.x) потребовала отладочной итерации и обновления зависимостей.

7.2 Что нового удалось извлечь

Скорость прохождения «скучных» этапов. Вайб-кодинг с Cursor/ИИ-ассистентом резко ускоряет создание типовых файлов: Pydantic-схемы, разводка роутов, конфиг БД — всё генерируется за секунды, высвобождая внимание для архитектурных решений и доменной логики.

Управляемый эксперимент. Стало проще итеративно менять форму congestion_index, структуру DTO или поведение алертов без часов ручного рефакторинга.

ИИ как «вторая голова». Ассистент помогает не упустить требования из ТЗ: необходимость RBAC, агрегированных метрик, сценариев мониторинга — всё находит отражение в коде благодаря диалоговому характеру разработки.

7.3 Степень успешности решения задачи

Сформированный прототип покрывает ключевые требования варианта CityPulse на уровне учебной реализации: реализованы сущности дорожных сегментов, потоков, алертов и метрик; предоставлен API для приёма данных и формирования агрегированной аналитики; веб-интерфейс с интерактивной картой отображает актуальные алерты и метрики.

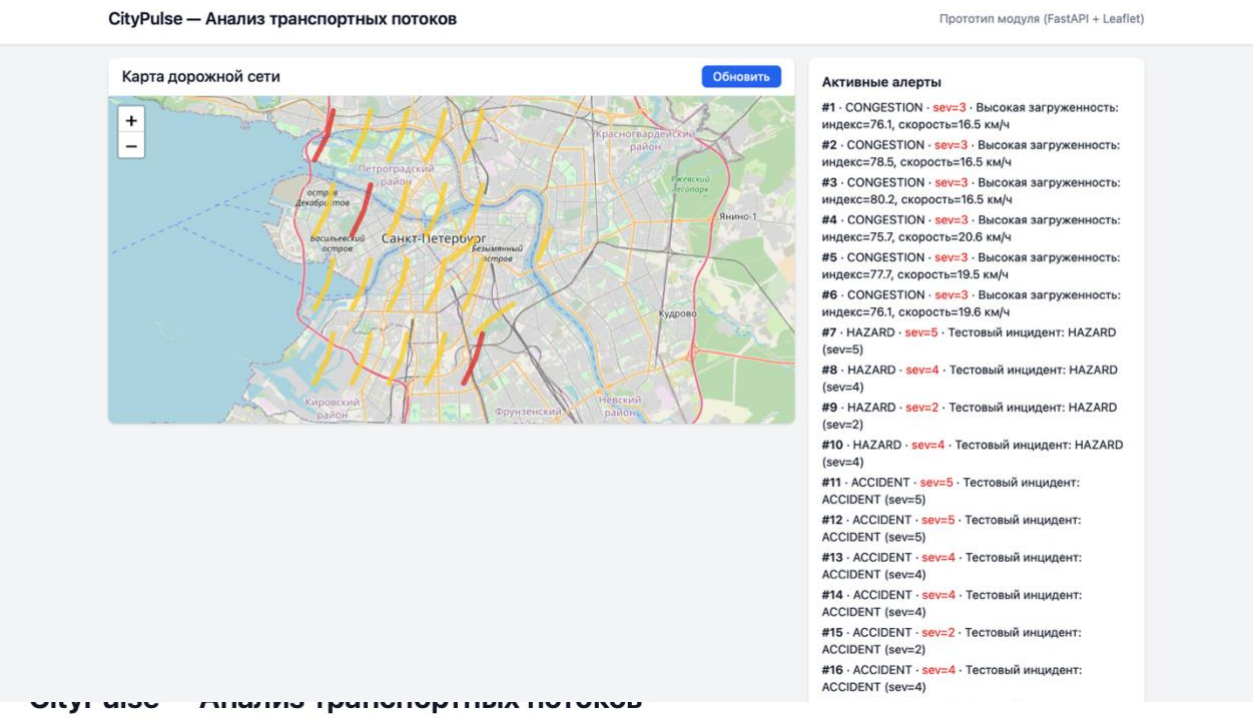
Хотя отсутствуют Kafka, PostGIS и полноценная ML-модель, архитектура позволяет внедрить их на следующих итерациях: замена DATABASE_URL для PostgreSQL, подключение Kafka-consumer к /api/traffic/ingest, добавление ML-модели в services_analytics.py — всё это не меняет публичный API. Задача по созданию и анализу сервиса в парадигме вайб-кодинга выполнена успешно.

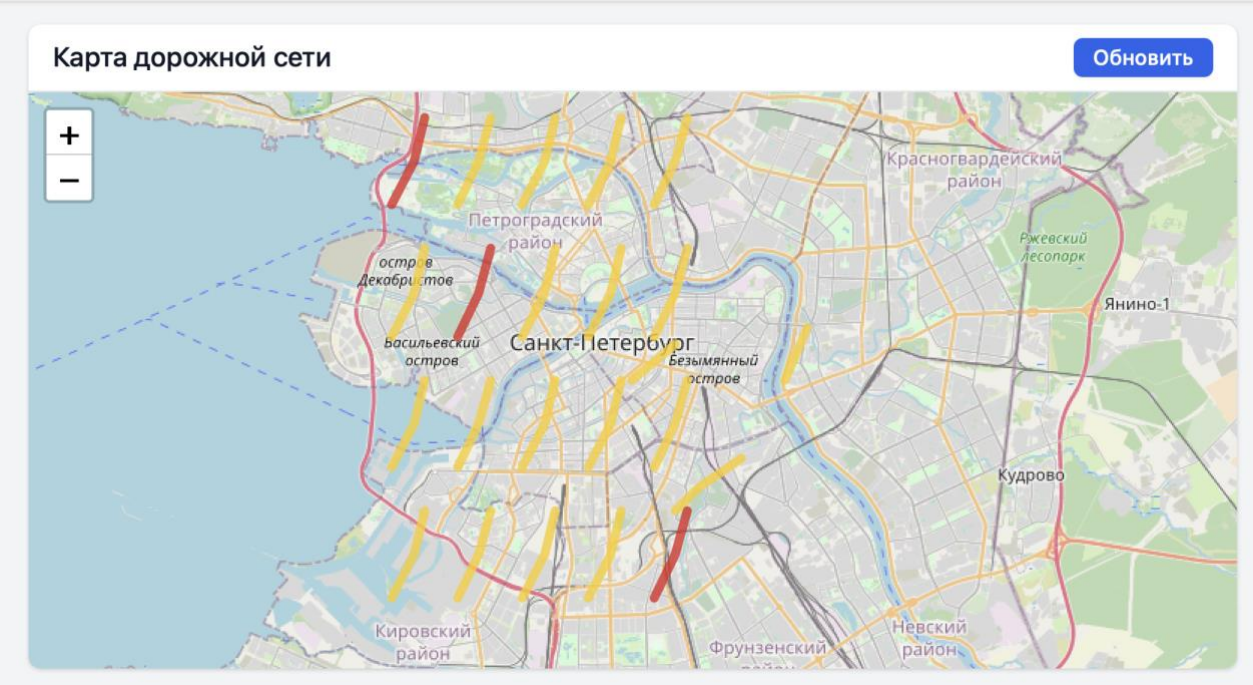
8. Заключение

В рамках лабораторной работы был спроектирован и реализован рабочий прототип модуля анализа и визуализации транспортных потоков системы CityPulse. Применение подхода вайб-кодинга (Cursor + ИИ-ассистент) позволило в сжатые сроки получить структурированный, расширяемый код с чётким разделением слоёв, работающим API и интерактивным картографическим интерфейсом.

Выявленные в ходе тестирования проблемы (несовместимость пакетов, пустая карта без тестовых данных, атрибуция Leaflet) были оперативно устранены итеративными правками через диалог с ИИ — что наглядно демонстрирует ключевое преимущество вайб-кодинга: быстрый цикл обратной связи между замыслом и работающим кодом.

9. Приложение


CityPulse — Анализ транспортных потоков
Прототип модуля (FastAPI + Leaflet)
Карта дорожной сети
Обновить
Активные алерты
#1 · CONGESTION · sev=3 · Высокая загруженность: индекс=76.1, скорость=16.5 км/ч
#2 · CONGESTION · sev=3 · Высокая загруженность: индекс=78.5, скорость=16.5 км/ч
#3 · CONGESTION · sev=3 · Высокая загруженность: индекс=80.2, скорость=16.5 км/ч
#4 · CONGESTION · sev=3 · Высокая загруженность: индекс=75.7, скорость=20.6 км/ч
#5 · CONGESTION · sev=3 · Высокая загруженность: индекс=77.7, скорость=19.5 км/ч
#6 · CONGESTION · sev=3 · Высокая загруженность: индекс=76.1, скорость=19.6 км/ч
#7 · HAZARD · sev=5 · Тестовый инцидент: HAZARD (sev=5)
#8 · HAZARD · sev=4 · Тестовый инцидент: HAZARD (sev=4)
#9 · HAZARD · sev=2 · Тестовый инцидент: HAZARD (sev=2)
#10 · HAZARD · sev=4 · Тестовый инцидент: HAZARD (sev=4)
#11 · ACCIDENT · sev=5 · Тестовый инцидент: ACCIDENT (sev=5)
#12 · ACCIDENT · sev=5 · Тестовый инцидент: ACCIDENT (sev=5)
#13 · ACCIDENT · sev=4 · Тестовый инцидент: ACCIDENT (sev=4)
#14 · ACCIDENT · sev=4 · Тестовый инцидент: ACCIDENT (sev=4)
#15 · ACCIDENT · sev=2 · Тестовый инцидент: ACCIDENT (sev=2)
#16 · ACCIDENT · sev=4 · Тестовый инцидент: ACCIDENT (sev=4)

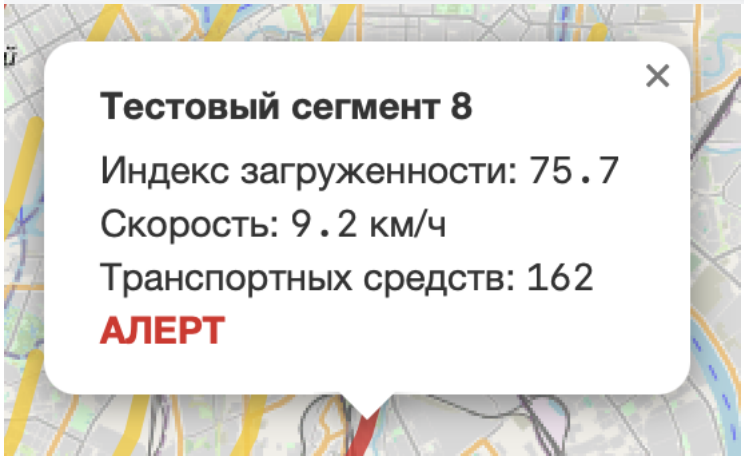

Карта дорожной сети
Обновить
+
−
Петроградский район
Красногвардейский район
Ржевский лесопарк
Янино-1
Кудрово
Кировский район
Фрунзенский
Невский район

Последние метрики

Сегмент 25: индекс=66.0, средний поток=111.0

Сегмент 24: индекс=67.3, средний поток=129.7

Сегмент 23: индекс=63.8, средний поток=108.7

Сегмент 22: индекс=43.6, средний поток=86.5

Сегмент 21: индекс=57.1, средний поток=112.2